Wrocław
University
of Science
and Technology

Software Engineering

Design patterns & Modern Architecture

Marcin Jodłowiec

January, 14th, 2026

Wrocław University of Science and Technology

Agenda

- Introduction
  - SOLID Principles
- Design Patterns
  - Selected Patterns
- From Patterns to Architecture
- Clean Code & DDD
- Modern Architectural Approaches

Wrocław
University
of Science
and Technology

Introduction

Motivation

Why pure Object-Orientetion is not enough

Typical symptoms of naive OOP projects:

- **Tight coupling** between classes.
- So-called **"God objects"** knowing and doing too much.
- **Mixed concerns**: Business logic tangled with infrastructure (user interface, database, communication).

As a result we end up with **code that works, but resists change**.

- How to deal with that?

Foundations

SOLID Principles

Before discussing patterns, we must understand the principles of good object-oriented design.

- **S – Single Responsibility Principle**: A class should have one, and only one, reason to change. One task, one purpuse, etc.
- **O – Open/Closed Principle**: Entities should be open for extension, but closed for modification.
- **L – Liskov Substitution Principle**: Subtypes must be substitutable for their base types.
- **I – Interface Segregation Principle**: Many client-specific interfaces are better than one general-purpose interface. Client should not be forced to implement unnecssary methods that they don't use.
- **D – Dependency Inversion Principle**: Depend upon abstractions, not concrete concepts.

Wrocław
University
of Science
and Technology

Design Patterns

Design Patterns

What are they?

Design Pattern: Definition

A design pattern is a **proven and reusable solution** to a **recurring software design problem** that:

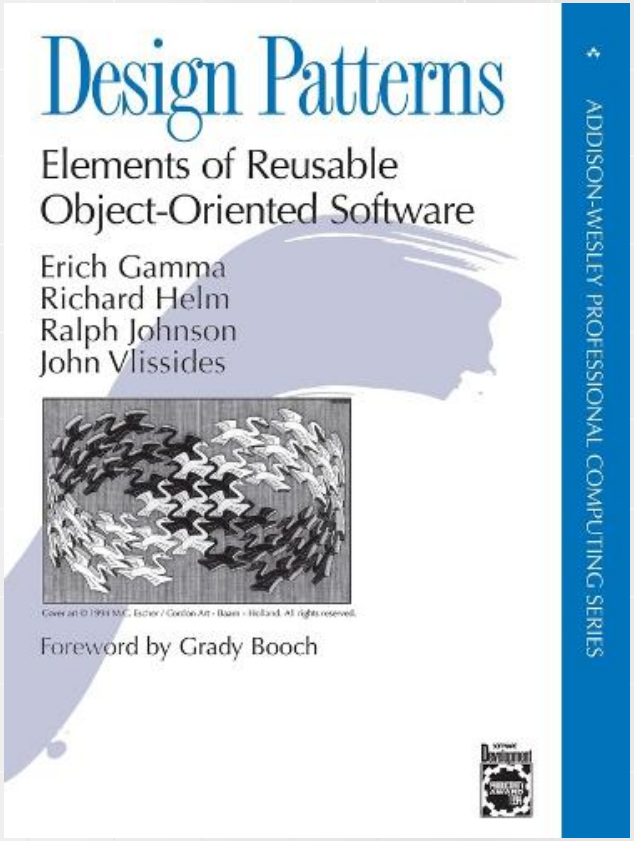
- applies in a **specific design context**
- describes a **general solution**, not a concrete implementation
- captures **best practices** derived from experience
- provides a **shared vocabulary** for design communication
- focuses on **structure and interaction** of software components

- Not ready-made solutions (copy-paste code).
- Not a library of classes.
- **A shared language** for communicating design decisions.
- Proven solutions to common problems in a specific context.

Famous Gang of Four (Gamma et al.)

- What is this famous GoF (Gamma, Helm, Johnson, Vlissides)?

Erich Gamma et al. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Massachusetts: Addison-Wesley. ISBN: 0-201-63361-2



Design Patterns

Classification (GoF)

- **Creational**
  - Control object creation
  - Examples: Singleton, Factory
- **Structural**
  - Organize class/object composition
  - Examples: Adapter, Decorator
- **Behavioral**
  - Assign responsibilities and communication
  - Examples: Observer, Strategy

Selected Patterns

Singleton

Intent: Ensure that a class has **only one instance** and provide a **global access point** to that instance.

Problem: Some objects (e.g. configuration managers, loggers) must be unique. Uncontrolled creation of instances may lead to **inconsistent state** or **resource conflicts**.

Solution (Idea): Restrict object creation by:

- making the constructor **private**,
- storing a **single static instance**,
- exposing a **static access method**.

Consequences

- ✓ Controlled access to a single shared instance
- ✓ Lazy initialization (optional)
- × Introduces **global state** (hidden dependencies)
- × Makes **testing and mocking harder**
- × Often violates the **Single Responsibility Principle**

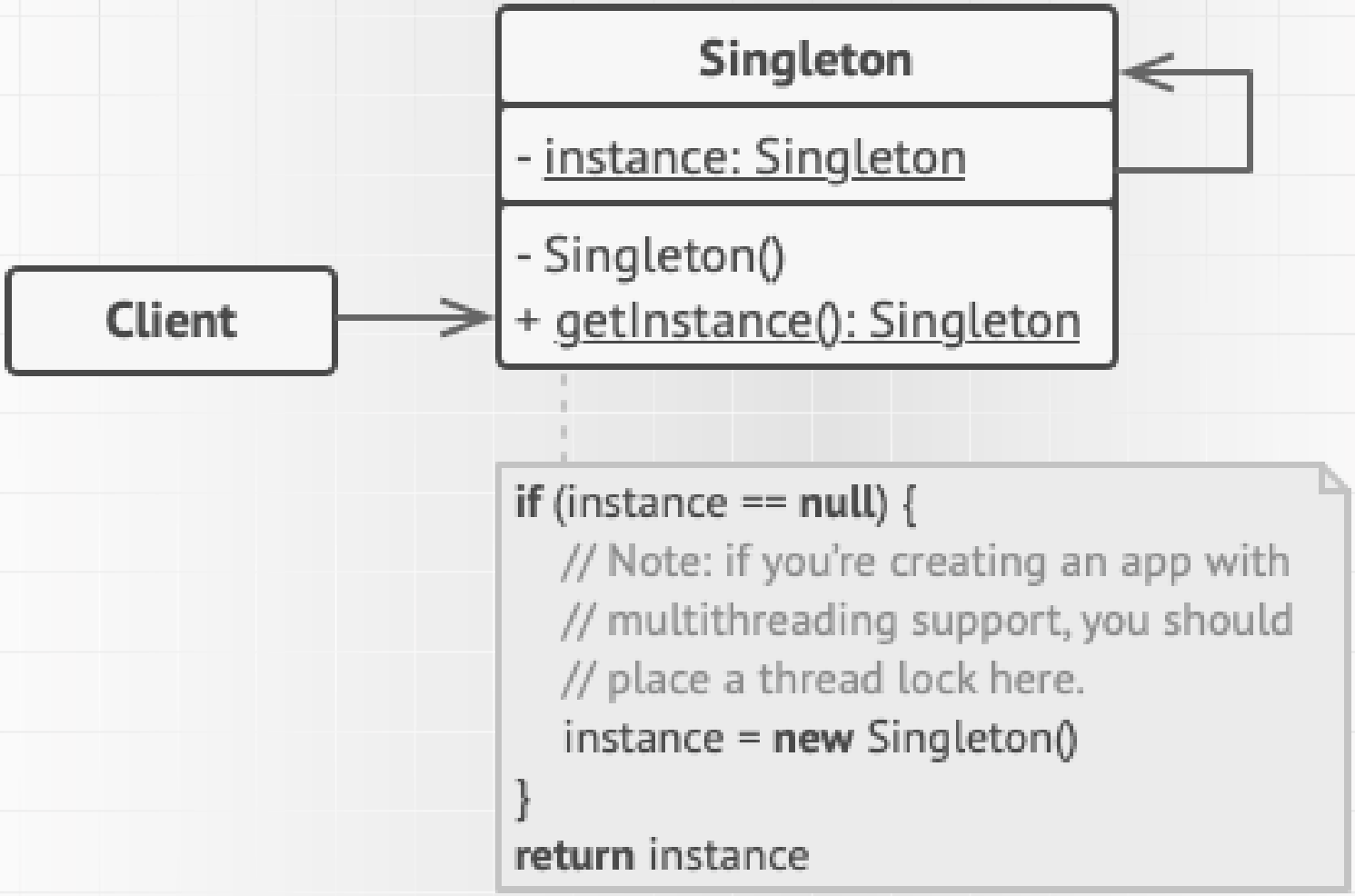


Figure: Source: refactoring.guru

- Avoid singletons, prefer **dependency injection** (you will see what is it in a few slides).

Selected Patterns

Factory / Factory Method

Intent: Define an interface for creating objects, while allowing subclasses to decide which concrete class to instantiate.

Problem: Direct object creation using new tightly couples the client to concrete classes, making the system **harder to extend** and **maintain**.

Solution: Encapsulate object creation by:

- defining a **factory interface or method**,
- delegating the choice of the **concrete class** to subclasses,
- returning objects through a **common abstraction**.

Consequences

- ✓ Separates **creation logic** from **usage logic**
- ✓ Supports the **Open/Closed Principle**
- ✓ Improves **extensibility** and **testability**
- × Introduces **additional classes** and indirection
- × May increase overall **code complexity**

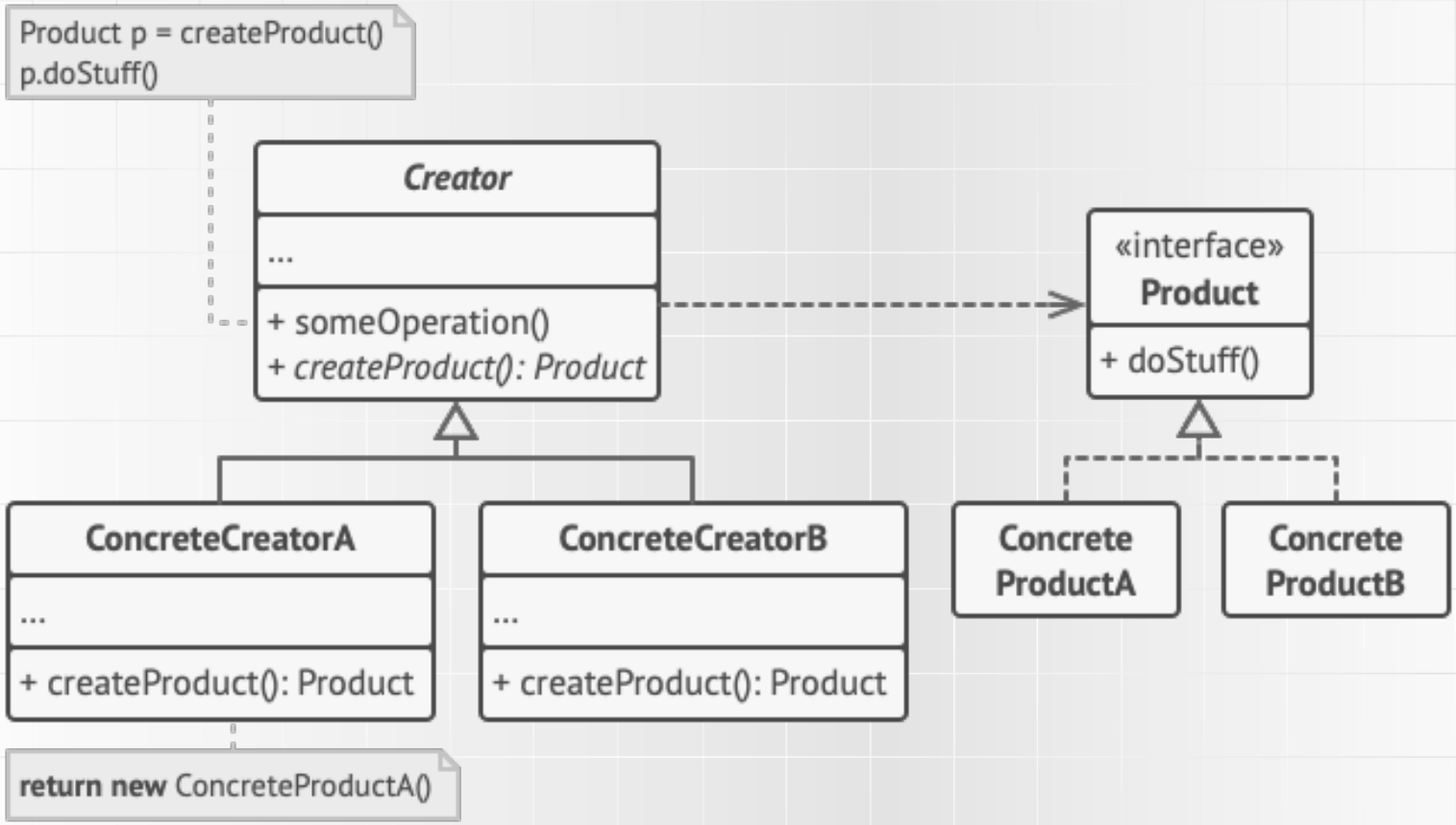


Figure: Source: refactoring.guru

Selected Patterns

Adapter

Intent: Convert the interface of an existing class into another interface expected by the client, allowing incompatible classes to work together.

Problem: Existing (often legacy or third-party) classes expose interfaces that are **incompatible** with the client code, and cannot be modified directly.

Solution: Introduce an adapter that:

- implements the **target interface** expected by the client,
- internally wraps the **adaptee** (existing class),
- translates requests from the client into calls understood by the adaptee.

Consequences

- ✓ Enables integration of **legacy** and **third-party** components
- ✓ Decouples the client from concrete implementations
- ✓ Preserves the **Single Responsibility Principle**
- × Adds an extra layer of **indirection**
- × May increase the number of classes in the system

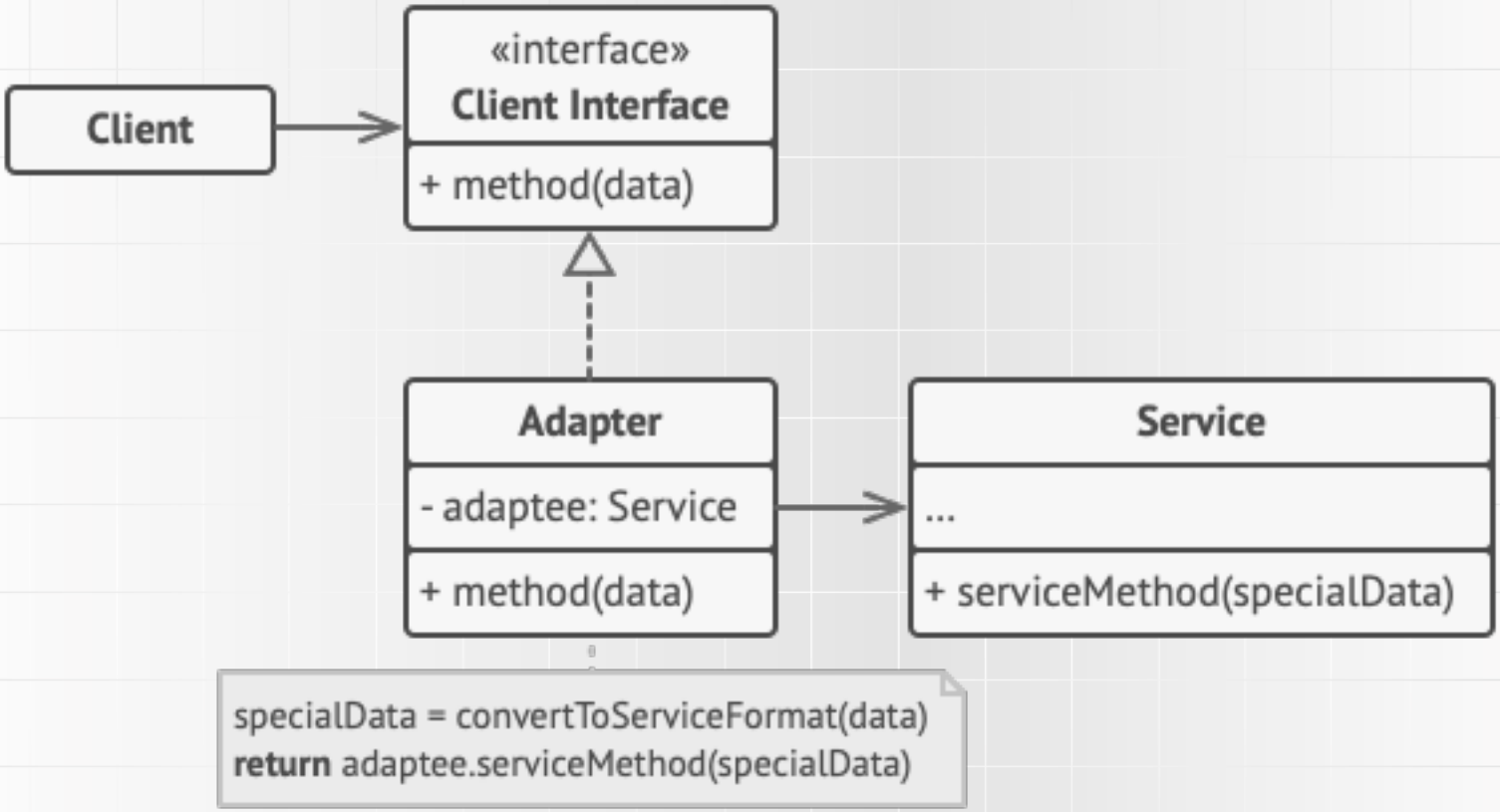


Figure: Source: refactoring.guru

Selected Patterns

Decorator

Intent: Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending behavior.

Problem: Extending behavior using inheritance leads to a **class explosion** and rigid designs, especially when behavior combinations are required.

Solution: Wrap objects using decorators that:

- implement the same **component interface**,
- hold a reference to a **wrapped object**,
- add behavior **before and/or after** delegating calls.

Consequences

- ✓ Supports the **Open/Closed Principle**
- ✓ Enables **dynamic composition** of behavior
- ✓ Favors **composition over inheritance**
- × Introduces additional **objects and indirection**
- × Can make the system **harder to understand and debug**

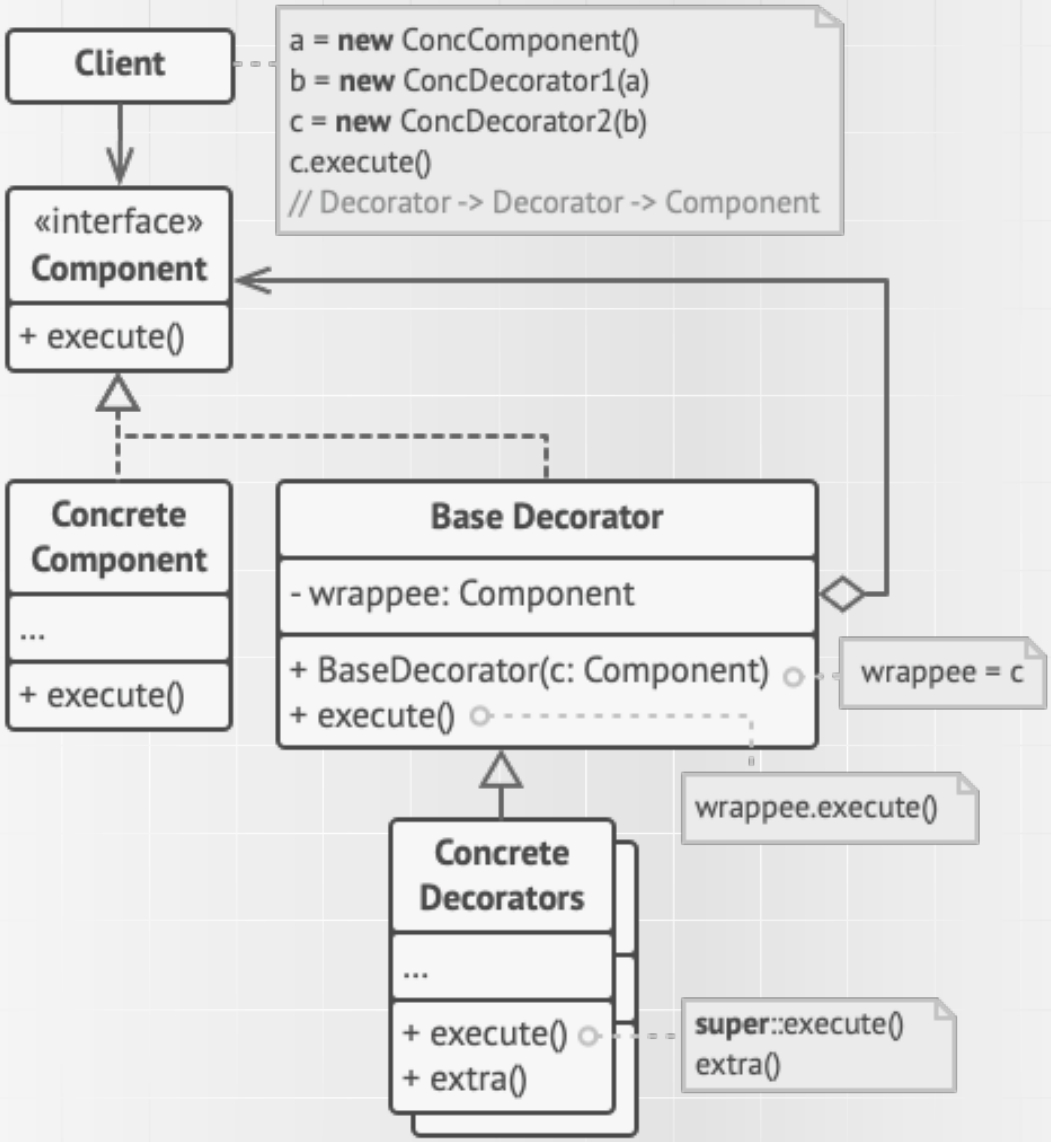


Figure: Source: refactoring.guru

- **Decorator** changes behavior and Adapter changes the interface.

Selected Patterns

Observer

Intent: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Problem: Multiple objects depend on the state of another object, but tight coupling would make the system **hard to extend** and **maintain**.

Solution: Introduce a subject that:

- maintains a list of **observers**,
- provides methods to **attach** and **detach** observers,
- automatically **notifies** observers when its state changes.

Consequences

- ✓ Enables **reactive and event-driven** behavior
- ✓ Promotes **loose coupling** between subject and observers
- ✓ Supports the **Open/Closed Principle**
- × May lead to **memory leaks** (Lapsed Listener Problem)
- × Can cause **unexpected update chains**

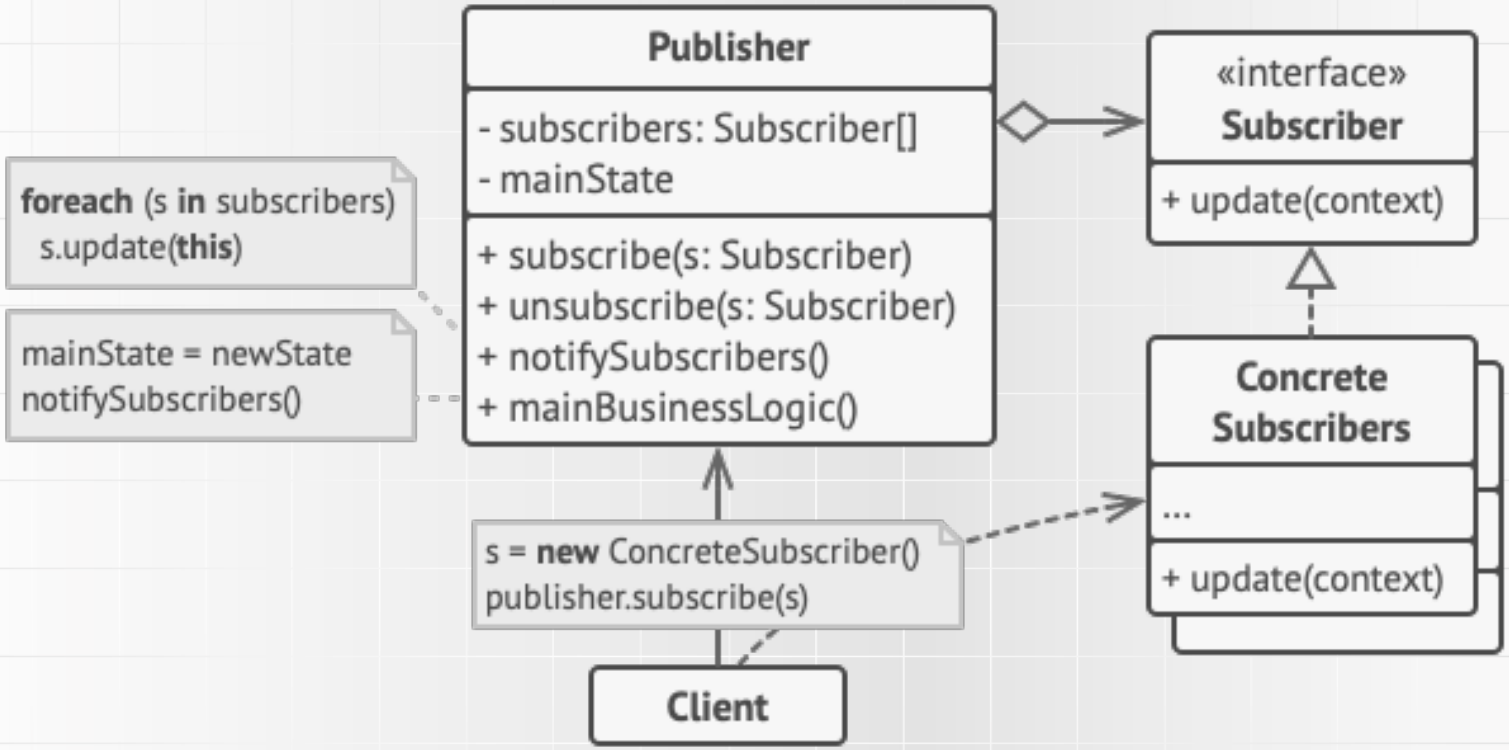


Figure: Source: refactoring.guru

Selected Patterns

Strategy

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable so that the algorithm can vary independently from its clients.

Problem: Multiple algorithmic variants implemented using conditional logic (`if/else`, `switch`) lead to **rigid** and **hard-to-maintain** code.

Solution: Encapsulate algorithms by:

- defining a common **strategy interface**,
- implementing each algorithm in a **separate class**,
- delegating algorithm selection to the **context**.

Consequences

- ✓ Eliminates complex conditional statements
- ✓ Allows **runtime switching** of algorithms
- ✓ Supports the **Open/Closed Principle**
- ✓ Improves **testability** of algorithms
- × Increases the number of **classes** in the system

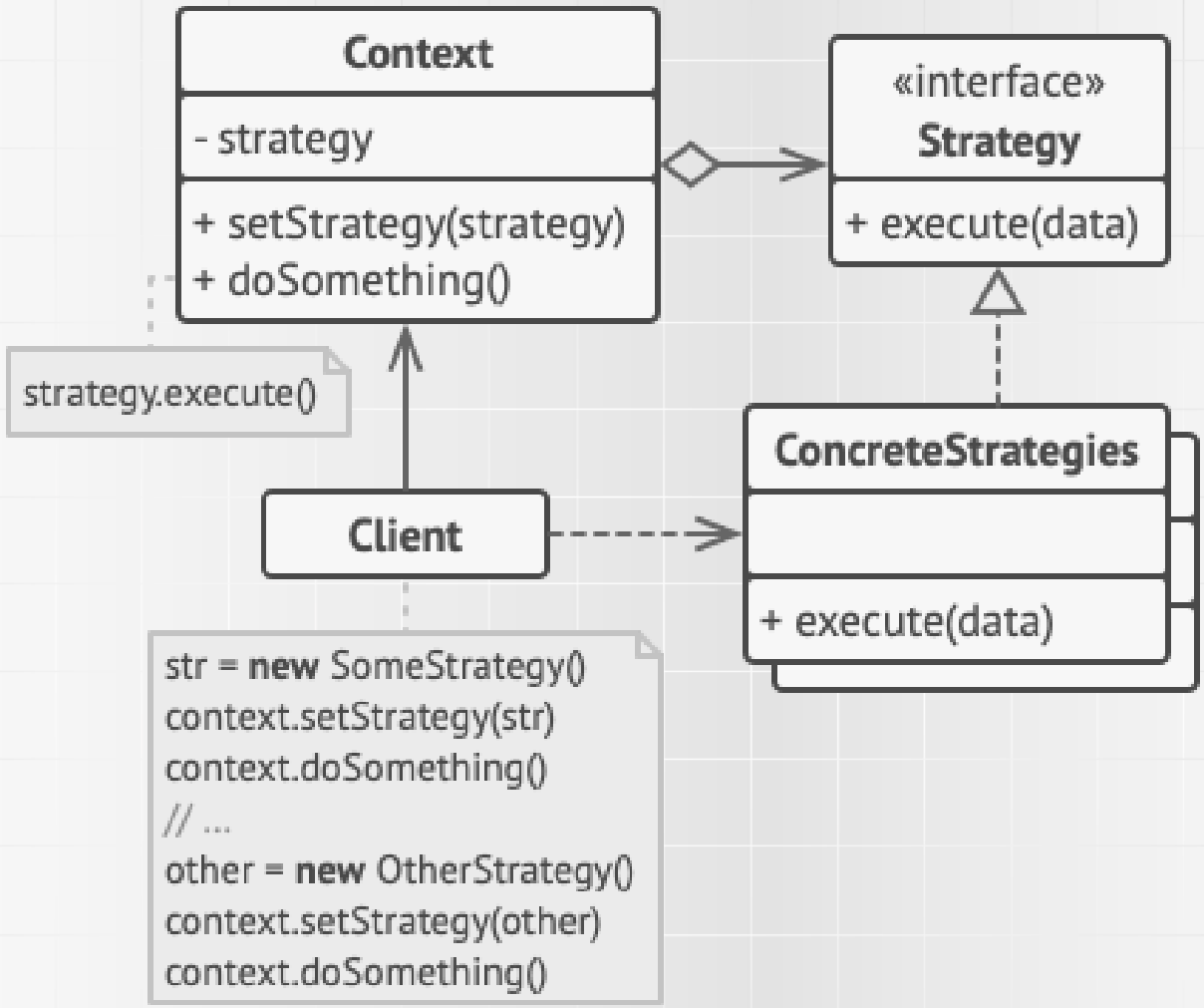


Figure: Source: refactoring.guru

Wrocław
University
of Science
and Technology

From Patterns to Architecture

Limits of Design Patterns

- GoF OOP design patterns improve quality **locally**, but they do not solve architectural problems.

Patterns work at the level of classes and objects. They do not address:

- Overall system structure.
- Layer dependencies.
- Module responsibilities.

Design vs Architecture

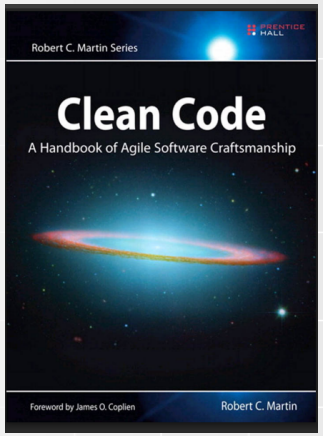
Design Pattern	**Architectural Pattern**
Local problem	Global decision
Classes / Objects	Modules / Layers / Components
Implementation detail	High-level structure
e.g., Strategy, Factory	e.g., MVC, Microservices, Hexagonal

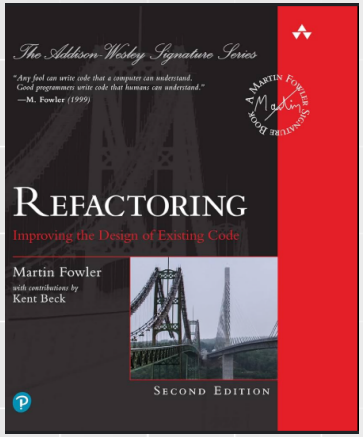
Wrocław
University
of Science
and Technology

Clean Code & DDD

Clean Code – Prerequisite for good architecture

- Good architecture requires good code.





- Code easy to read and comprehensible
- Easy to change
- Tests, naming convention, exceptions, formatting, comments
- „code smells” → bad code heuristics (M.Fowler)
  - long method, god object, duplication, primitive obsession, shotgun surgery

Domain-Driven Design (DDD)

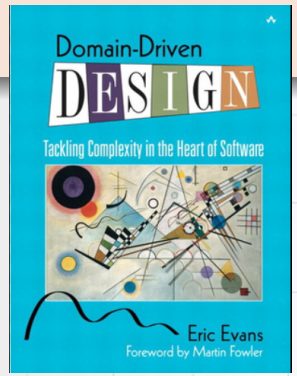
Core Concepts

Domain-Driven Design (DDD)

Domain-Driven Design is an approach to software development that focuses on modeling the **core business domain** and expressing it explicitly in the code.

DDD emphasizes:

- a **rich domain model** that reflects business concepts,
- a **shared language** between developers and domain experts (Ubiquitous Language),
- clear **boundaries** separating the domain from technical concerns.



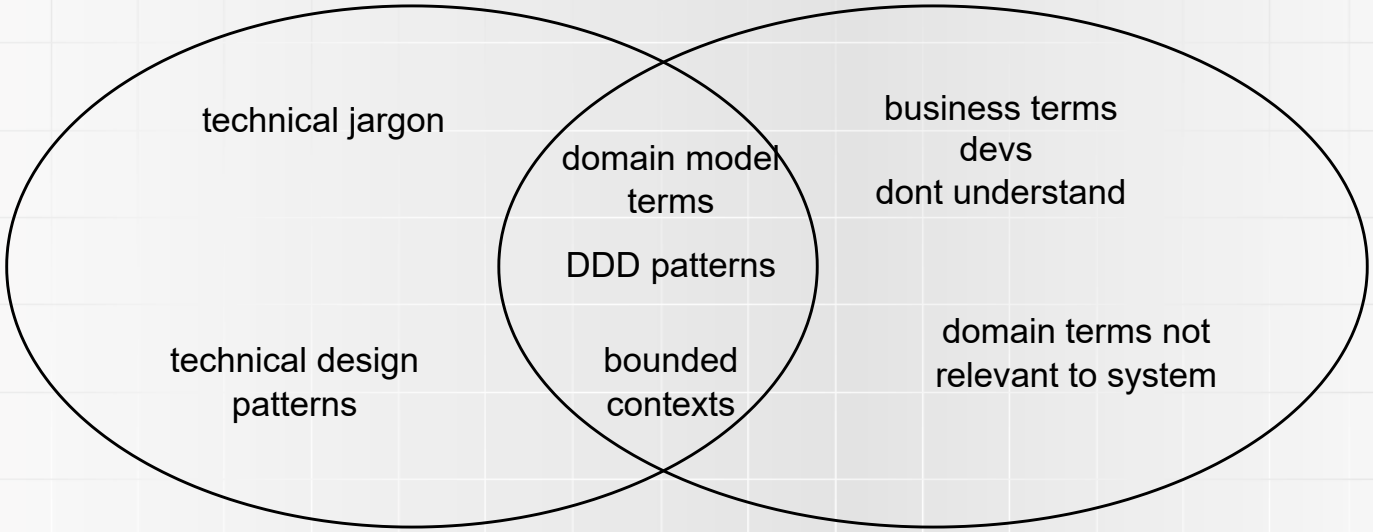


Figure: Ubiquitious Language

DDD Building Blocks

- **Entity**: Defined by identity, not attributes (e.g., User with ID).
- **Value Object**: Defined by attributes, immutable, no identity (e.g., Money, Address).
- **Aggregate**: A consistency boundary that groups entities and value objects and enforces domain invariants.
- **Aggregate Root**: The single entry point to an aggregate; all external access goes through the root.
- **Bounded Context**: Explicit boundary within which a domain model is defined and applicable.

Within a bounded context:

- terms have a **precise and shared meaning**,
- the model is **logically unified**,
- code and data follow the same conceptual rules.

Outside the boundary, **different models may apply** and **explicit translation** is required.

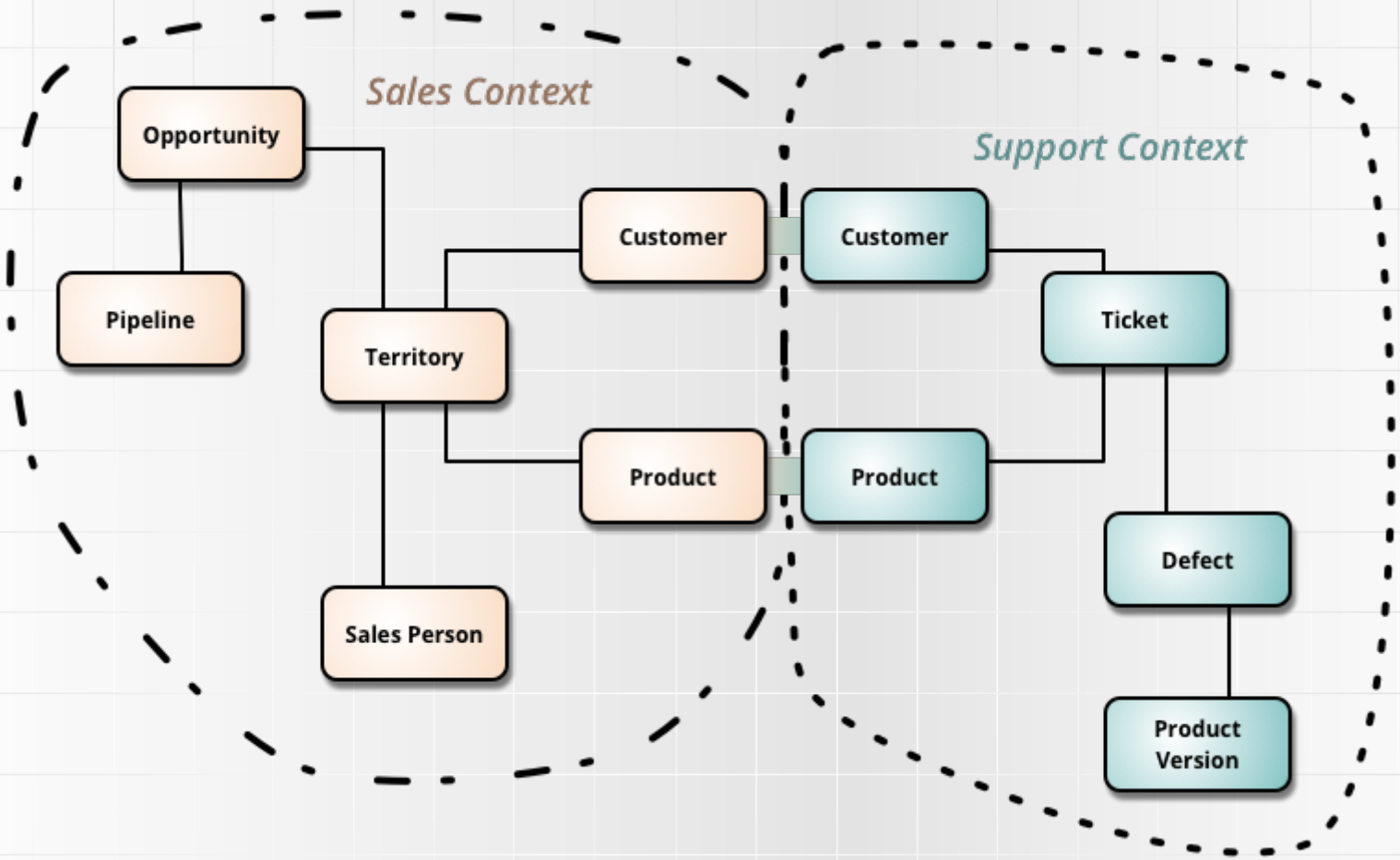


Figure: Source: M. Fowler

- Notice **Product** and **Customer** – appear in both contexts because they represent different domain concepts with different meanings, even though they share the same name.

Domain-Driven Design

Strategic vs Tactical

Aspect	Strategic DDD	Tactical DDD
Primary focus	System structure and boundaries	Domain model and code structure
Main question	*Where does a model apply?*	*How is the model implemented?*
Key concepts	Bounded Context, Context Map	Entity, Value Object, **Aggregate**, Repository
Level	Architecture, teams, integration	Classes, methods, invariants
Typical decisions	Context boundaries, integration style	Encapsulation, consistency rules
Failure mode	Conceptual chaos, mixed models	Anemic domain model

Wrocław University of Science and Technology

Modern Architectural Approaches

Modern Architectural Approaches

Clean Architecture

Clean Architecture is an architectural approach that aims to **separate business logic from technical concerns** and **protect the core domain from external dependencies**.

Key principles

- Dependencies always point **inward**
- Business rules are **independent of frameworks**
- The core domain is **testable in isolation**

Typical layers

- **Entities** — enterprise business rules
- **Use Cases** — application-specific rules
- **Interface Adapters** — controllers, presenters
- **Frameworks & Drivers** — UI, database, external systems

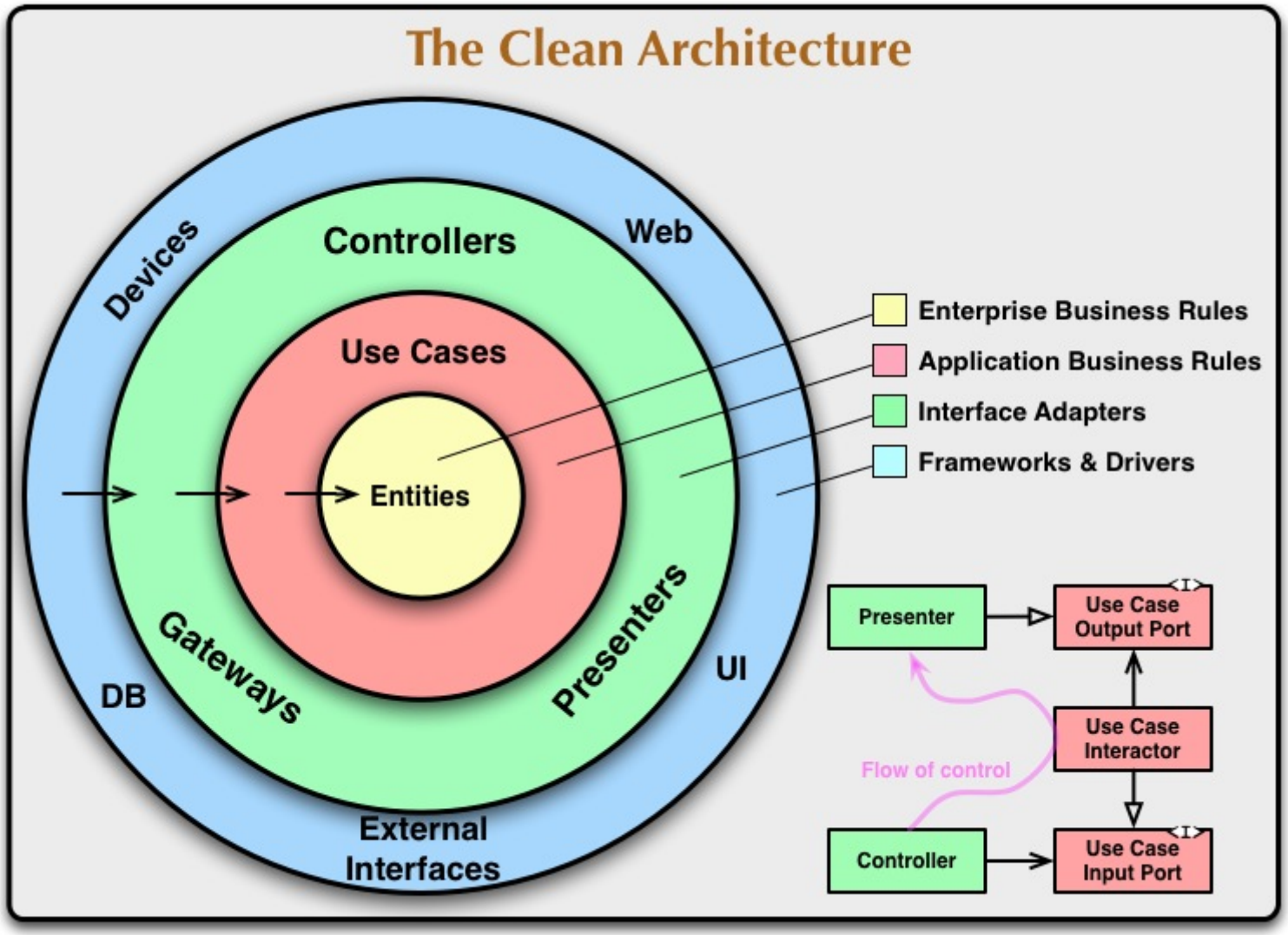


Figure: Source: R. Martin

Command Query Responsibility Segregation (CQRS)

CQRS is an architectural pattern that separates **write operations (commands)** from **read operations (queries)**.

Motivation

- Different requirements for **writing** and **reading** data.
- Complex domain logic on writes, performance-oriented reads.
- Avoiding overly complex and overloaded domain models.

Core idea

- **Commands** change system state (no return value).
- **Queries** return data (no side effects).
- Separate models for reads and writes.

Consequences

- ✓ Clear separation of responsibilities.
- ✓ Better scalability and performance tuning.
- × Increased complexity and duplication.
- × Harder consistency management.

CQRS

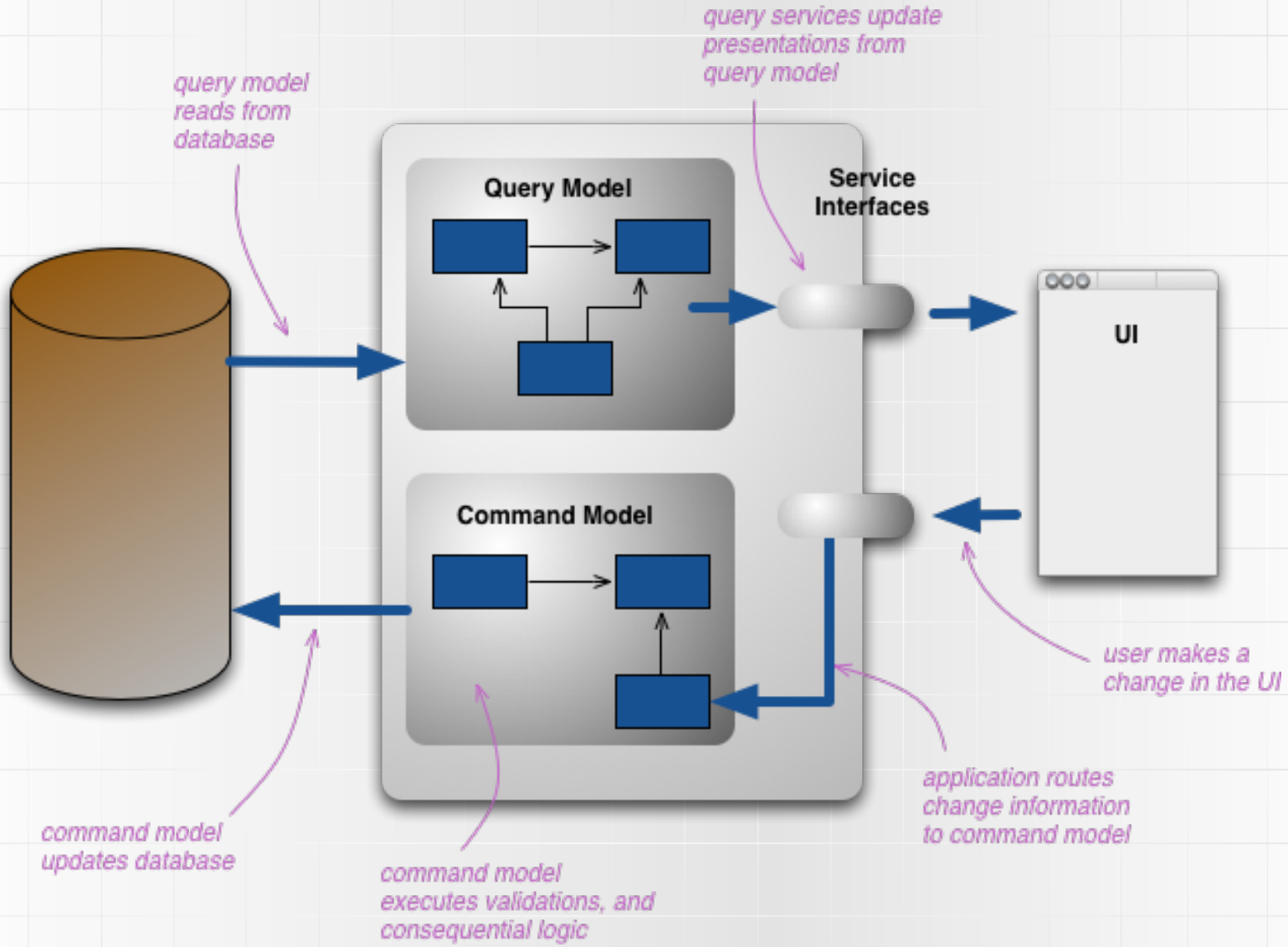


Figure: Source: M Fowler

Dependency Injection (DI)

Dependency Injection: definition

A design technique in which an object *receives* its dependencies instead of creating them.

Core idea: *Inversion of Control* – separation of **object construction** from **object behavior**.

- Dependencies are expressed as **interfaces**, not concrete implementations.
- Improves **testability** (substitution, mocking, isolation).
- Enables **loose coupling** and modular architecture.
- **Frameworks** (e.g. Spring, .NET, Angular) often manage object creation and wiring via *containers*.

Dependency Injection

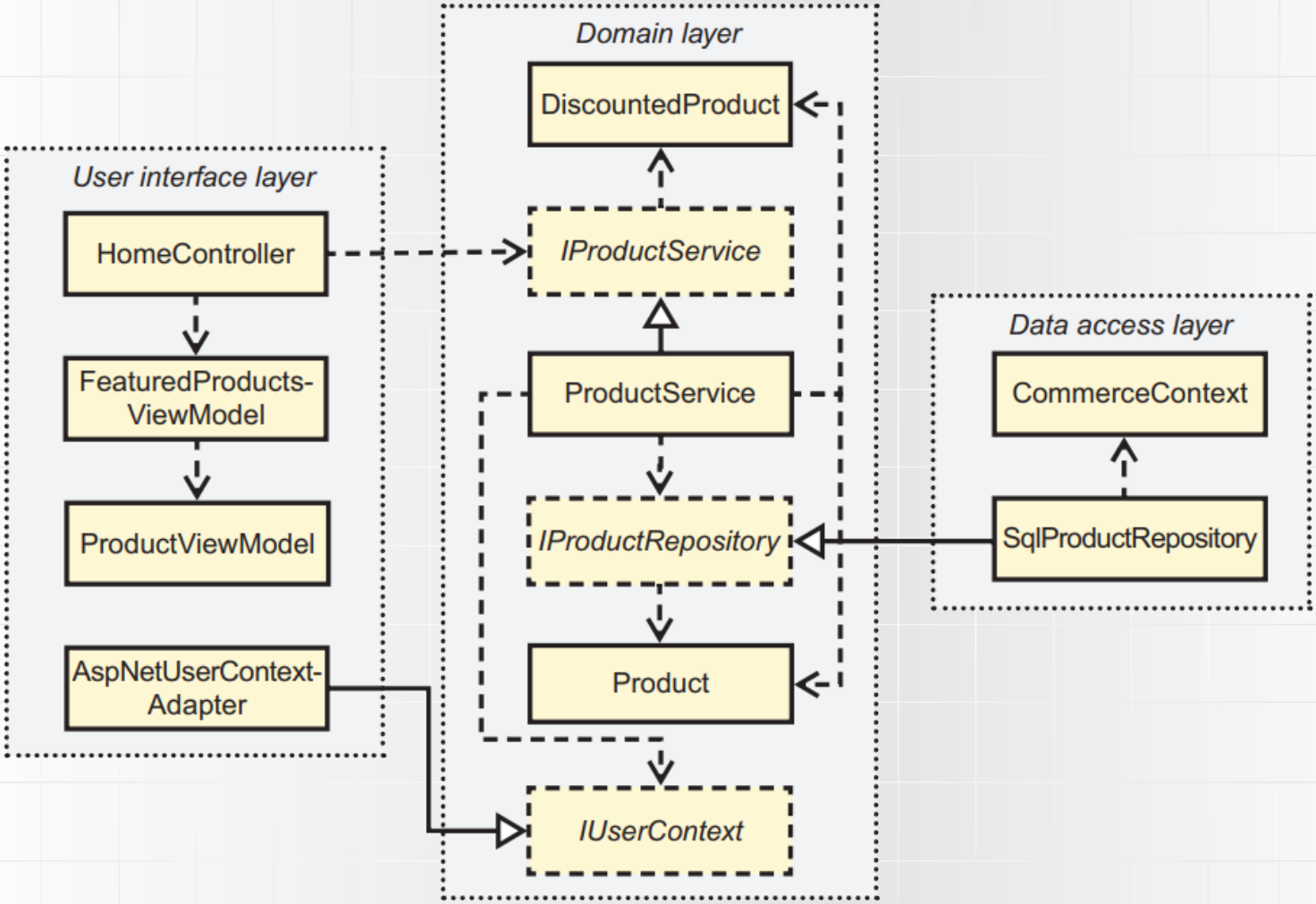


Figure: Source: Mark Seemann, Steven van Deursen - Dependency Injection Principles, Practices, and Patterns, Manning Publications, 2019

Wrocław University of Science and Technology

It's all for today!